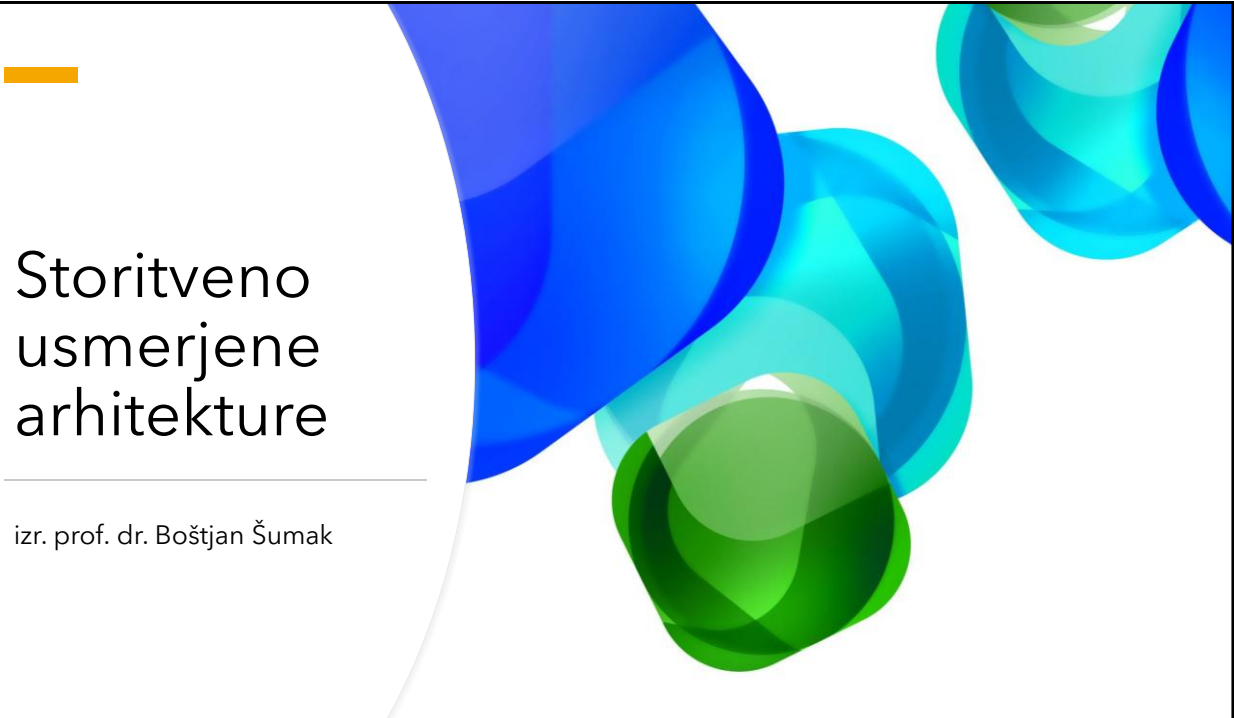




Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak

1

Nazadnje

- Razvoj storitev REST
 - Java (JAX-RS, SpringBoot)
 - .NET (ASP.NET Core, ASP.NET Core MinimalAPI)
 - JavaScript (express + mongoose)
- Tehnologije za opis tehničnih vmesnikov storitev

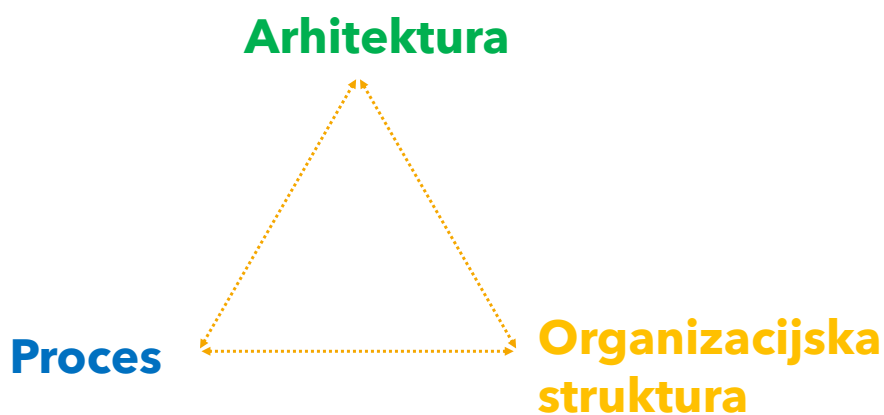
2

Arhitekture mikrostoritev

3

(že govorili...)

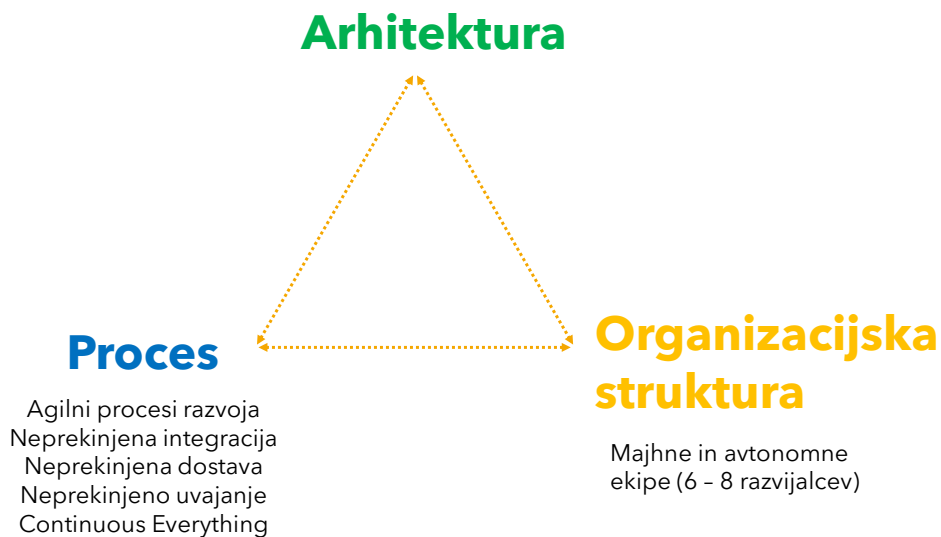
Temelji za uspešen razvoj programske opreme



4

(že govorili...)

Temelji za uspešen razvoj programske opreme



5

Arhitektura?

6



7

Monolitna arhitektura

- Je arhitektura, v kateri je programska oprema načrtovana, da **deluje kot ena, samo-vsebovana enota**
- Pogosto uporabljena pri načrtovanju tudi sodobnih rešitev
- Komponente v monolitni arhitekturi so povezane med različnimi nivoji in med seboj močno odvisne
 - rezultat je **tesna sklopljenost programske kode**

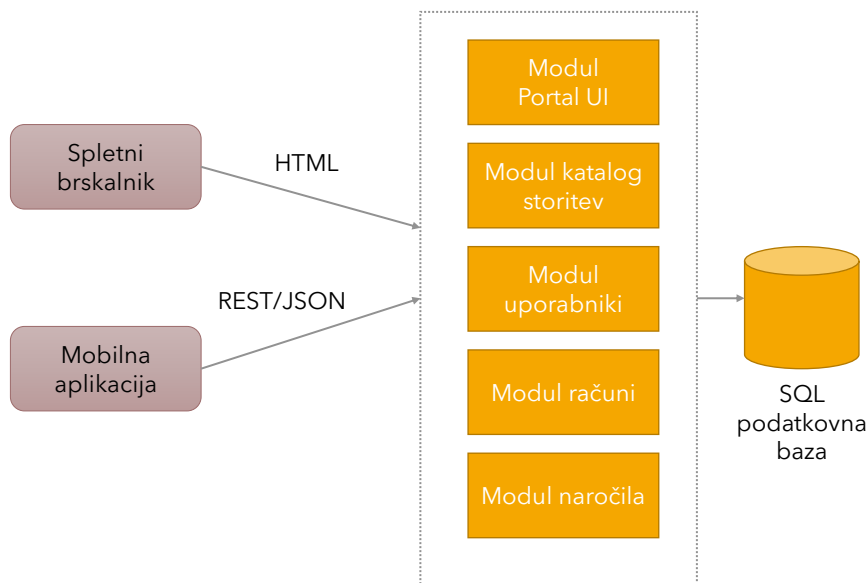
8

Lastnosti monolitnih aplikacij

- Poslovna logika je pogosto **modularna**
 - storitve, domenski objekti, dogodki, ...
- Aplikacija običajno zgradi, zapakira in namesti kot ena celota (monolit)
 - Način namestitve je odvisen od jezika/ogrodja (npr. en proces, en paket, en deployable artefakt).
- Skaliranje načeloma ni težava
 - podvajanje instanc + load balancer (horizontalno skaliranje)

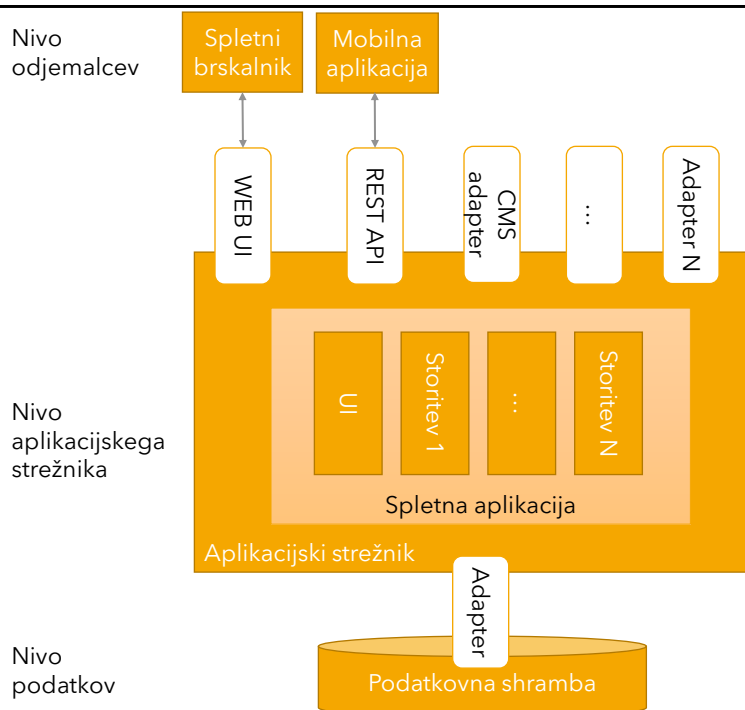
9

Primer rešitve na osnovi monolitna arhitekture



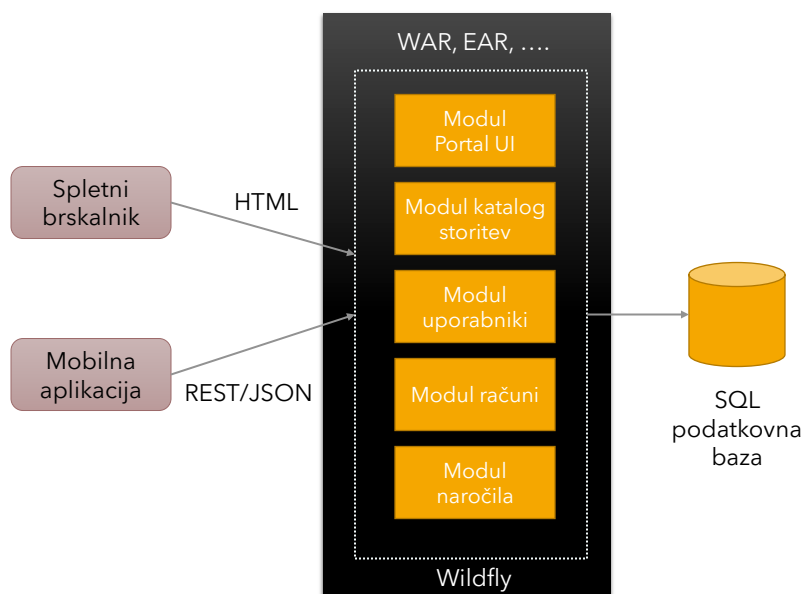
10

Monolitna večnivojska arhitektura spletne aplikacije



11

Primer rešitve na osnovi monolitne arhitekture



Ko razvijamo rešitev na osnovi monolitne arhitekture, pomeni, da na koncu vse module zapakiramo skupaj.

V Javi se zgradi datoteka WAR, EAR, mogoče tudi JAR.

Podobno je v drugih programskih jezikih pri izgradnji namestitvenih paketov.

12

Join at menti.com | use code 6251 1071

Mentimeter

Prednosti razvoja na osnovi monolitne arhitekture?

N cors tezav

Hitrejši razvoj

Manj odvisnosti

lažje razumevanje za manjše projekte

Vse imamo na enem mestu, skupna baza..

hitrejši razvoj, vse na enem mestu Manj CORS težav

Enostavnejše zaganjanje sam ne rabimo skrbeti za neskončno povezav

težje skaliranje

6 / 15

13

Prednosti razvoja na osnovi monolitnih arhitektur

- Enostaven razvoj**
 - arhitektura je praviloma dobro poznana večini razvijalcev, sistem je preglednejši.
- Preprosto nameščanje**
 - aplikacija se običajno izdelava in namesti kot enoten namestitveni paket (en artefakt).
- Enostavno testiranje**
 - po zagonu so vse funkcionalnosti na voljo v isti aplikaciji, kar olajša integracijsko in sistemsko testiranje.
- Enostavna ponovna uporaba kode**
 - ker je vse v istem paketu oz. zbirki knjižnic, ki so povezane, je ponovno uporabo modulov in skupnih komponent preprostejša.
- Dobra podpora razvojnih okolij**
 - IDE praviloma omogočajo učinkovito razvijanje, razhroščevanje (debugiranje) in refaktoriranje monolitnih rešitev.

14



Vse je super

dokler...

15

Join at menti.com | use code 62511071

Mentimeter

Izzivi z monolitnimi aplikacijami?

Skaliranje

Prevelik projekt

Kompleksnost se veča ko raste obseg projekta

Obseg kode, hitro se zgodi podvajanje

Veliko uporabnikov in skaliranje

Težje vzdrževanje koda, ko se poveča kompleksnost.

ni za velike produkcijske aplikacije

Vzporedni razvoj

Preobremenjenost omrežja

Ločevanje odgovornosti po posameznih servisih

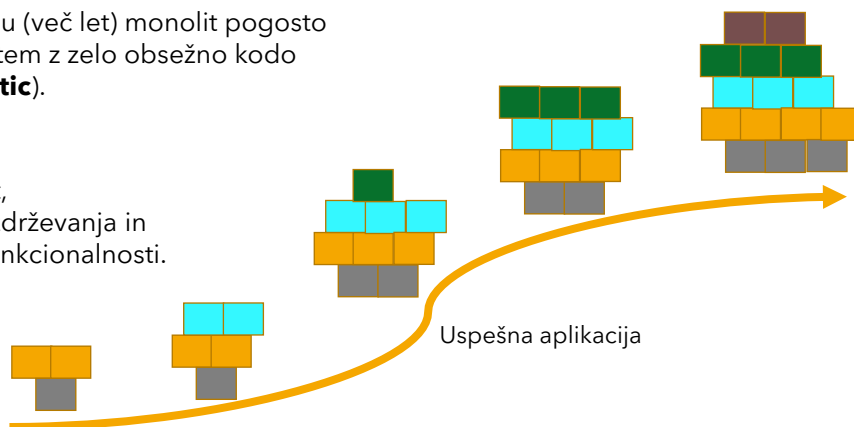
← 🔍 → Next slide

7/15

16

Izzivi z monolitnimi aplikacijami?

- Uspešne aplikacije se nenehno razvijajo, dopolnjujejo in širijo – razvoj se po prvi dostavi praviloma ne konča.
 - Ekipe vsakodnevno dodajajo novo funkcionalnost, zato obseg kode hitro narašča.
- Pri dolgoročnem razvoju (več let) monolit pogosto preraste v zelo velik sistem z zelo obsežno kodo (tudi **več milijonov vrstic**).
- Posledice rasti:
 - slabša preglednost,
 - večja zahtevnost vzdrževanja in
 - težji razvoj novih funkcionalnosti.



17

Monolitna arhitektura : izzivi s pogostim nameščanjem

- **Pogosto nameščanje sprememb monolitnih aplikacij postane težavno**
 - Vsaka sprememba zahteva **namestitev celotne aplikacije**, ne le dela.
 - Deploy lahko **prekine dolgo-izvajajoče procese** v ozadju.
 - Večji monolit pomeni **večje tveganje napak** in daljši čas izpada delovanja.
- Zato se ekipe pogosto **izogibajo pogostemu nameščanju**.

1,797 files changed +10424 -55796 lines changed

MY COWORKERS
WATCHING ME DEPLOY A
"SMALL FIX" ON A FRIDAY



18

Monolitna arhitektura : izzivi s skaliranjem aplikacij

- V aplikaciji imamo lahko module, ki
 - zahtevajo veliko pomnilnika
 - zahtevajo veliko procesorske moči, ali
 - izvajajo operacije, ki izvajajo zahtevne IO operacije
- Zelo hitro pridemo do izziva **kako izbrati ustrezno programsko in strojno opremo za učinkovito skaliranje aplikacij**
 - za zagotavljanje zadostne količine virov (pomnilnik, procesor, IO), modulom, ki potrebujejo več virov

19

Velikost aplikacij → učinkovitost razvoja

Velika aplikacija lahko močno upočasni nalaganje in delo v razvojnem okolju (IDE)

upočasnjen razvoj v IDE

+

počasno ponovno nalaganje na aplikacijskem strežniku (nekaj minut)

=

MANJŠA PRODUKTIVNOST

Velika aplikacija lahko signifkantno poveča čas, ki je potreben za zagon na aplikacijskem strežniku

izgublja se kontrola nad delovnim tokom,
veliko je motenj v procesu razvoja in
veliko časa porabimo za neproduktivne stvari

20

Monolitna arhitektura: **dolgoročna vezava na določen sklad tehnologij**

- Pri gradnji rešitev na osnovi monolitne arhitekture se običajno izbere in na dolgi rok uporablja sklad izbranih tehnologij
- Monolitna aplikacija = veliko kode, spisane na osnovi izbranih tehnologij
→ pisanje in komponent na osnovi novih tehnologij je
 - **drago**
 - **tvegano**
 - pogosto **nemogoče**
- Posledica: **rešitve na osnovi monolitne arhitekture pogosto uporabljajo zastarele tehnologije**

21

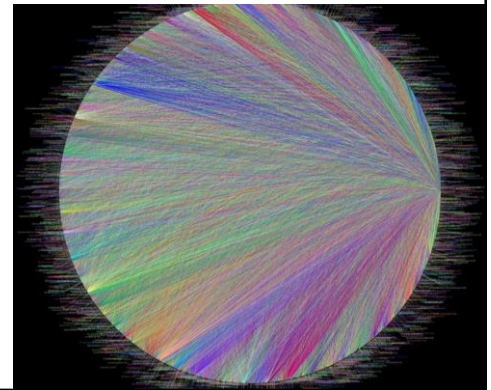
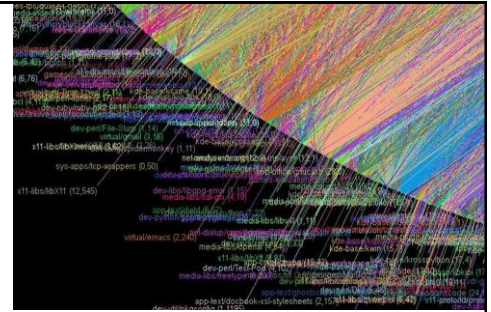
Monolitna aplikacija se širi in postaja vse bolj kompleksna, kar **na dolgi rok prinaša izzive:**

- **Razvoj in nameščanje** (uvajanje v okolje) velikih monolitov postaneta zahtevnejša.
- **Preglednost in vzdrževanje** se poslabšata - kodo je težje razumeti, spreminjati in nadgrajevati.
- **Vzporedno delo več ekip** je težje, kar lahko upočasni iterativni (agilni) razvoj.
- **Testiranje postane počasnejše**, saj je spremembe potrebno preverjati na ravni celotne aplikacije.
- **Odpravljanje napak in dodajanje funkcionalnosti** zahtevata vse več časa - tveganje za neželene stranske učinke naraste.
- **Izdaja posodobitev** pogosto vključuje zahtevno usklajevanje in združevanje sprememb (integracija).
- **Posledice za zmogljivost:** večje aplikacije se praviloma dlje zaganjajo in zahtevajo več virov.

22

Monolitne aplikacije=nočna mora razvijalcev?

- Izzivi so tudi **organizacijski** - vsak razvijalec prispeva k isti aplikaciji, zato je **avtonomija ekip omejena**
 - Spremembe se prepletajo**, kar poveča **potrebo po usklajevanju** in oteži neodvisno načrtovanje ter izvedbo.
- Skalabilnost in zanesljivost je težje optimizirati**
 - vire je mogoče praviloma dodeljevati le celotni aplikaciji, ne posameznim delom.
- Sčasoma lahko monolit zdrsne v t. i. »**monolitni pekel**«
 - sistem postane prevelik in prekompleksen za učinkovito vzdrževanje.

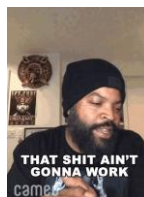


23

Monolitna Arhitektura

Proces

~~Agilni~~
~~Neprekinjena integracija~~
~~Neprekinjena dostava~~
~~Neprekinjeno uvajanje~~



Organizacijska struktura

~~Majhne in avtonomne~~
 ekipe

24

Rešitev?

razvoj na osnovi **ARHITEKTURE MIKROSTORITEV**



25

Join at menti.com | use code 62511071

Mentimeter

Kaj je arhitektura mikrorstitev?

Delitev monolita v "minilite" -> manjše dele

arhitektura z več mikro storitvami

Razdelitev aplikacije na manjše neodvisne dele

Arhitektura kjer storitve poganjamo neodvisne druga od druge

Vec storitev se poveze v celoto. Vsaka posamezna storitev je majhna in odgovorna za neko specifcno zadevo. Prav tak vsaka mikrorstitev upravlja s svojimi podatki.

Vsaka storitev skrbi za svoje podatke

Lazje vzdrzevanje kode, uvajanje sprememb...

Locevanje odgovornosti, izoliranost, lazja kontrola nad projektom

← 🔍 → Next slide

6 / 15

26

Kaj je arhitektura mikrororitev?



27

Mikrororitev – izvor

- Izraz mikrororitev (ang. microservice) se je začel uporabljati na delavnici arhitektov programske opreme **maja 2011** (v okolici Benetk)
 - za poimenovanje nastajajočega arhitekturnega sloga
 - maja 2012 je ista skupina izraz »**microservices**« sprejela kot uradno poimenovanje.
- Prakse, podobne mikrororitvam, so obstajale že prej, pogosto pod drugimi imeni
 - Velika podjetja kot so Microsoft, Amazon, Netflix, Facebook so sicer že več let razvijala rešitve, ki sledijo slogu mikrororitev arhitekturne rešitve,
 - velikokrat so slog poimenovali po svoje (npr. **mikro spletne storitve**, **finozrnati SOA**, itd.)
- Širša prepoznavnost se je močno okrepila z zapisi in predstavitvami skupnosti (npr. **Fowler/Lewis**).

28

Mikrostoritve = **odgovor na naraščajočo kompleksnost?**

- Namesto ene velike monolitne aplikacije sistem razdelimo na **več manjših, funkcionalno zaokroženih storitev**.
- Posamezna mikrostoritev je **samostojna aplikacija** z jasno odgovornostjo.
- Mikrostoritev izpostavi **API** kot vmesnik za komunikacijo z drugimi storitvami ali z odjemalci.
- Mikrostoritve lahko v nekaterih pristopih vključujejo tudi del **uporabniškega vmesnika** (npr. po domenah ali funkcionalnostih).
- Izvajajo se kot **ločene instance** v lastnem izvajalnem okolju, pogosto v **oblaku** in/ali v **zabojnikih** (npr. Docker).

29

Mikrostoritev - definicija

„Samostojna storitev z jasno opredeljeno, ozko usmerjeno funkcionalnostjo, ki jo prek aplikacijskega programskega vmesnika (API) ponuja drugim delom sistema.“

- Običajno predstavlja **poslovno zmožnost** znotraj določene domene
 - npr. upravljanje uporabnikov, naročil, plačil ...).
- Pogosto je odgovorna za podatke svoje domene
 - ustvarjanje, branje, posodabljanje in brisanje (CRUD) podatkovnih zapisov, vezanih na eno (ali majhno skupino) povezanih entitet podatkovnega modela.

30

Mikrostoritev - primer

- Funkcionalnost **prejem zalog** implementiramo kot **samostojno mikrostoritev**.
- Zahtevek za vnos novih zalog prispe prek **HTTP** (API); pošlje ga lahko druga mikrostoritev ali spletna aplikacija (uporabniški vmesnik).
- Mikrostoritev **validira podatke** in zabeleži prejem v svojo podatkovno zbirko.
- Na koncu vrne **odgovor** (npr. potrditev uspešnega vnosa / identifikator zapisa), s katerim potrdi sprejem zalog.



31

Karakteristike mikrostoritev

Mikrostoritev je odgovorna za eno zmožljivost

Mikrostoritev je samostojno namestljiva

Mikrostoritev je sestavljena iz enega ali več procesov

Mikrostoritev ima lastno shrambo podatkov

Majhna ekipa lahko vzdržuje nekaj mikrostoritev

Mikrostoritev je zamenljiva

<https://livebook.manning.com/book/microservices-in-net-core/chapter-1/31>

32

Arhitekturni stil mikrostoritev

„An approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.”

Vir: James Lewis, Martin Fowler, <https://martinfowler.com/articles/microservices.html>

33

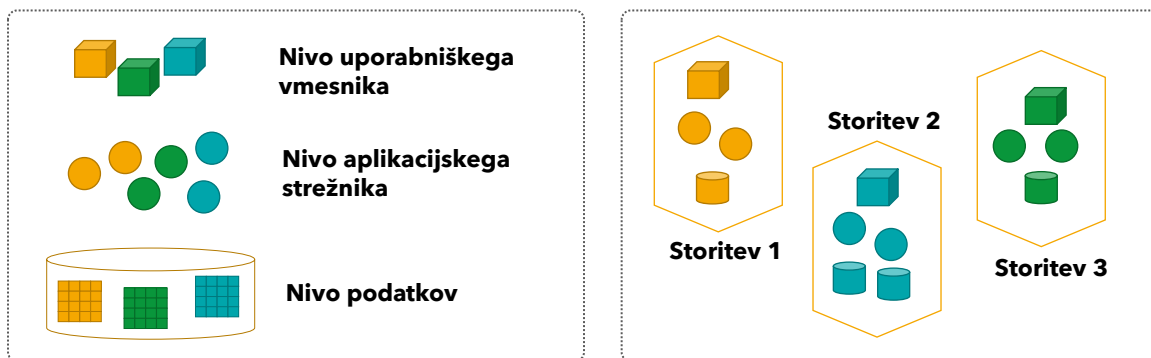
Arhitektura mikrostoritev (ali mikrostoritvena arhitektura)

- **Arhitekturni stil, ki omogoča izgradnjo aplikacij na osnovi zbirke majhnih in neodvisnih storitev z visoko stopnjo zmožnosti vzdrževanja in testiranja**
 - Vsaka storitev predstavlja **poslovno zmožnost**, ima **jasno odgovornost** in komunicira prek **dogovorjenih vmesnikov (API)**.
 - Ključna prednost je **večja evolutijska sposobnost sistema**: storitve je lažje razvijati, testirati in nameščati (uvajati) ločeno.
- James Lewis in Martin Fowler sta arhitekturni stil opisala z množico karakteristik

34

Arhitektura mikrostoritev

- Storitve (komponente aplikacije) se gradijo na osnovi **poslovnih zmožnosti**.
- Vsaka storitev se izvaja samostojno, pri čemer se v gruči zagotovi ustrezno število instanc.



35

Arhitektura mikrostoritev omogoča

- **Hitrejšo in pogostejšo dostavo** (delivery) in **večjo zanesljivost uvajanja sprememb**
 - tudi pri večjih in kompleksnih sistemih
- **Visoko skalabilnost ter učinkovito nameščanje in izvajanje** v oblaki infrastrukturi, pogosto z uporabo PaaS
- **Postopno posodabljanje tehnologij**
 - lažje uvajanje novosti in zamenjava zastarelih tehnologij po posameznih storitvah, brez potrebe po preoblikovanju celotnega sistema

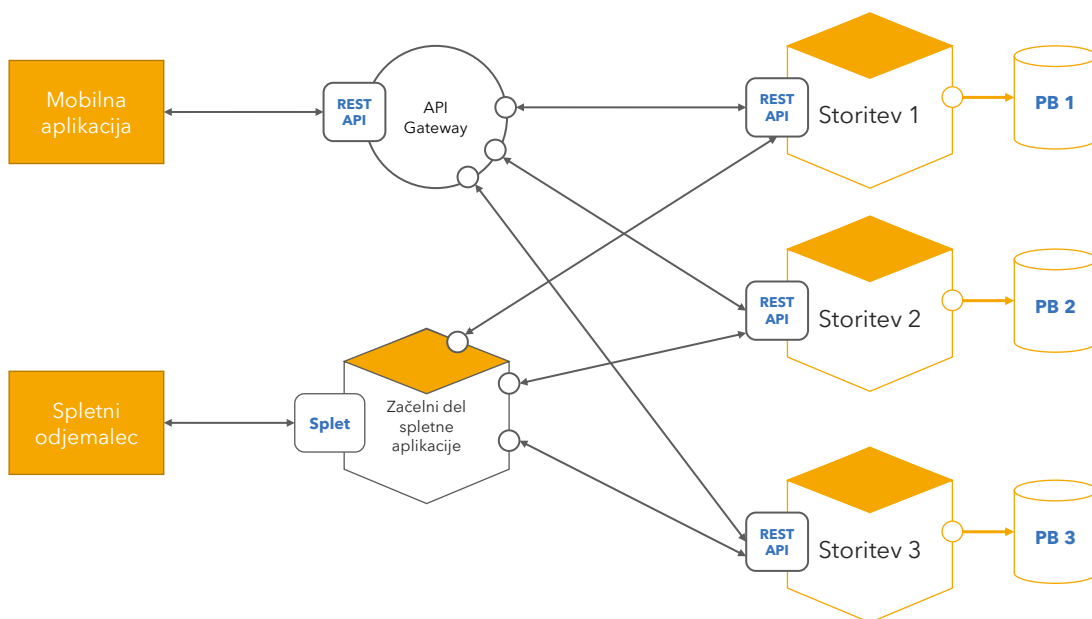
36

Arhitektura mikrostoritev

- Posamezne storitve morajo biti **neodvisne in učinkovito povezane**
 - Za **sinhrono komunikacijo** se najpogosteje uporablja **HTTP API** (npr. REST) ali gRPC.
 - Za **asinhrono komunikacijo** se pogosto uporabljajo **sporočilni sistemi** (npr. RabbitMQ) in dogodkovno usmerjen pristop.
- Praviloma se izogibamo centraliziranim komunikacijskim mehanizmom
 - npr. »storitevno vodilo« kot enotna točka integracije
 - integracija je porazdeljena med storitvami.

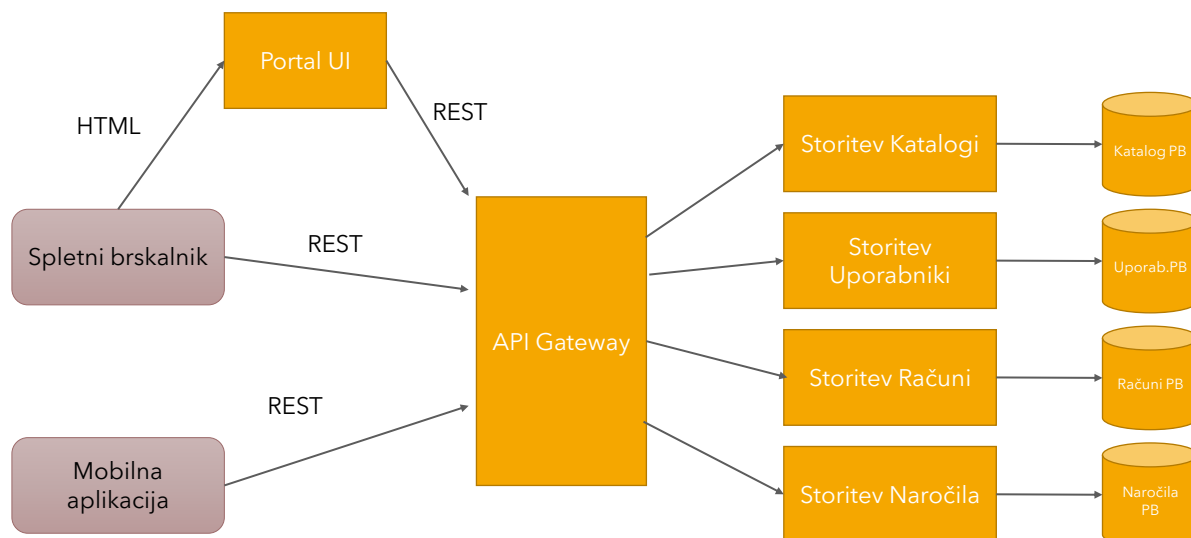
37

Arhitektura mikrostoritev



38

Primer rešitve na osnovi arhitekture mikrostoritev

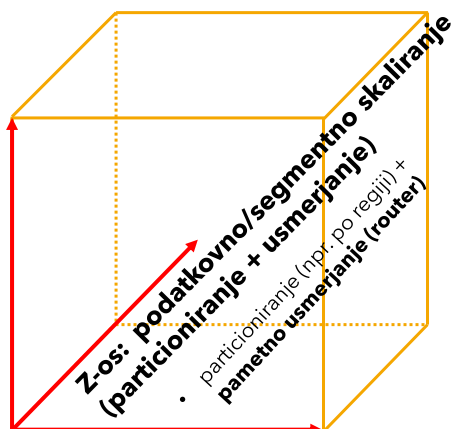


39

Arhitektura mikrostoritev: možnosti za skaliranje?

Y-os: funkcijsko skaliranje (funkcijska dekompozicija)

- Sistem razdelimo na različne funkcionalne dele/storitve
- Skaliramo tiste storitve, kjer je obremenitev največja



X-os: horizontalno skaliranje (kloniranje, replikacija)

- Povečamo število enakih instanc iste storitve (ali komponente)
- Promet porazdelimo prek load balancerja

Abbot, M., Fisher, M., Art of Scalability, The: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise

40

Mikrostoritvena

Arhitektura

Proces

Agilni
Neprekinjena integracija
Neprekinjena dostava
Neprekinjeno uvajanje
....



Organizacijska struktura

Majhne in
avtonomne
ekipe

41

Karakteristike mikrostoritvene arhitekture

42

Karakteristike arhitekture na osnovi mikrorstitev

Komponentizacija
na osnovi storitev

Organizacija kode
okrog poslovnih
zmožnosti

»Produkti«
namesto projektov

Pametne končne
točke in neumne
cevi

Decentralizirano
upravljanje
podatkov

Avtomatizacija
infrastrukture

Načrt za
neuspešno
delovanje

Evolucijsko
načrtovanje

Vir: <https://martinfowler.com/articles/microservices.html>

43

Mikrostoritvena arhitektura: Komponentizacija preko storitev

- J.L. in M.F.: ***komponenta je enota programske opreme, ki je neodvisno zamenljiva in jo je mogoče nadgrajevati ločeno.***
- V **tradicionalnih sistemih** so komponente pogosto organizirane kot **knjižnice**
 - vključene v program in uporabljene s klici funkcij znotraj istega procesa.
- V mikrostoritveni arhitekturi je osnovni način komponentizacije **razčlenitev sistema na storitve**
 - Vsaka storitev se izvaja v svojem procesu in z drugimi deli sistema komunicira prek **dogovorjenih vmesnikov** (API).

44

Mikrostoritvena arhitektura: **Komponentizacija preko storitev**

- Glavna prednost: **neodvisno nameščanje** (deploy) posameznih storitev.
- Pri arhitekturi na osnovi knjižnic sprememba komponente pogosto zahteva **ponovno izgradnjo in namestitev celotne aplikacije**.
- Pri mikrostoritvah lahko storitev praviloma **posodobimo ločeno**, brez vpliva na ostale.
- **Izziv:** sprememba **APIja** lahko zahteva prilagoditve odvisnih storitev.
Rešitev: jasne meje storitev + evolucija vmesnikov (npr. **verzioniranje API**, **združljivost za nazaj**).

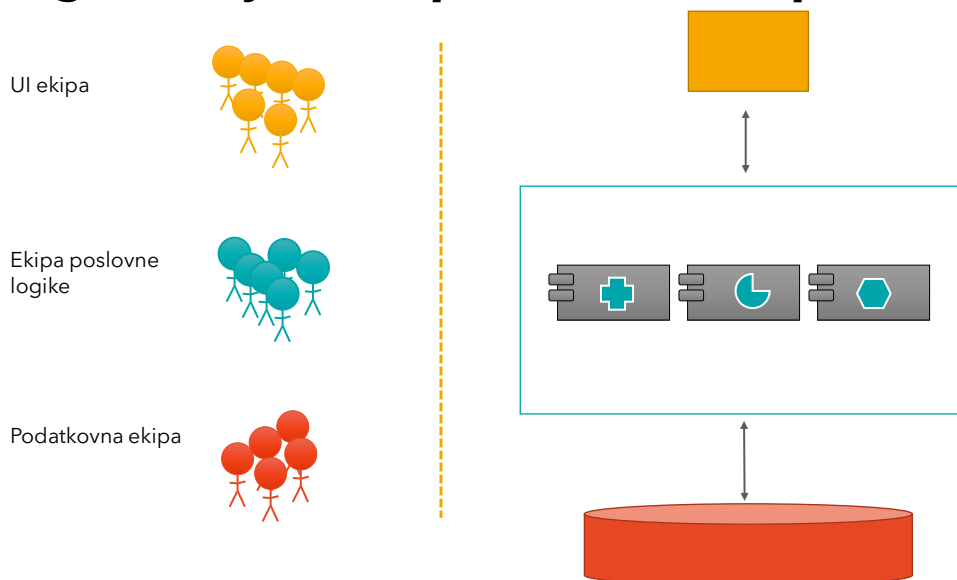
45

Monolitna arhitektura: **Organizacija kode po tehnoloških plasteh**

- V večjih monolitnih aplikacijah je koda pogosto organizirana po tehnoloških plasteh (»layered architecture«)
- Posledično so tudi ekipe pogosto razdeljene in organizirane glede na tehnološki vidik, npr.:
 - uporabniški vmesnik (UI)
 - poslovna logika / strežniški del (back-end)
 - podatkovni sloj / podatkovna baza
 - ...

46

Monolitna arhitektura: Organizacija kode po tehnoloških plasteh



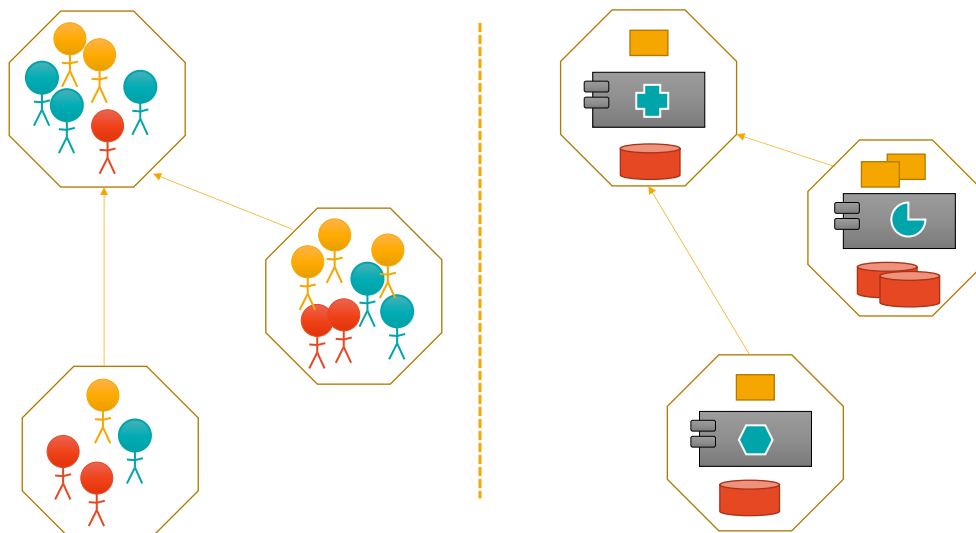
47

Mikrostoritvena arhitektura: Organizacija kode okrog poslovnih zmožnosti

- Pri mikrostoritvah je pristop drugačen: **kodo razdelimo na storitve glede na poslovne zmožnosti (domeno)** in ne glede na tehnološke plasti.
- **Implementacija posamezne storitve lahko vključuje tudi več slojev**, na primer:
 - uporabniški vmesnik (če je del storitve),
 - trajna hramba in upravljanje podatkov,
 - integracije z zunanjimi sistemi,
 - ipd.
- Tak pristop zahteva **večfunkcijske (cross-functional) ekipe**, ki pokrijejo celoten življenjski cikel storitve
 - npr. UX, podatkovno modeliranje, razvoj, testiranje, DevOps, ...

48

Mikrostoritvena arhitektura: Organizacija kode okrog poslovnih zmožnosti



49

Mikrostoritvena arhitektura: »Produkti« namesto projektov

- **Klasičen projektni pristop** pogosto pomeni, da se po zaključku razvoja rešitev preda v vzdrževanje, projektna ekipa pa se razpusti.
- Pri **mikrostoritveni arhitekturi** je poudarek drugačen: storitve (in rešitev kot celota) se obravnavajo kot **produkt**, ki se **neprestano razvija in izboljšuje**.
- Ekipa, ki rešitev razvija, je praviloma **lastnik produkta** skozi **celoten življenjski cikel** (razvoj, uvedba v uporabo, delovanje, izboljšave).
- To pomeni večjo **odgovornost za kakovost in delovanje tudi po uvedbi** (»you build it, you run it«).

50

Mikrostoritvena arhitektura: »**Produkti**« namesto projektov

- **Miselnost produkta** izhaja iz poslovnih potreb:
 - rešitve ne razumemo kot seznam funkcionalnosti, ki jih je treba »dokončati«,
 - temveč kot izdelek, ki se **stalno razvija in izboljšuje** ter uporabnikom pomaga dosegati **poslovne cilje**.
- Zato ekipe **neprekinjeno spremljajo uporabo**, merijo učinke in iščejo načine, kako rešitev **poveča vrednost za uporabnike**.
 - Tak pristop je mogoč tudi pri monolitnih rešitvah, vendar ga mikrostoritve pogosto podprejo bolj naravno.
- Ker so storitve **manjše in jasno razmejene**, je komunikacija med razvijalci in uporabniki storitev (odjemalci) praviloma preprostejša, **odgovornosti so jasneje določene**, povratne informacije pa hitreje vodijo v izboljšave.

51

Mikrostoritvena arhitektura: **Pametne končne točke & neumne cevi**

- V klasičnih arhitekturah SOA je komunikacijska infrastruktura pogosto zgrajena okoli storitvenega vodila (ESB), ki ponuja napredne funkcije
 - **usmerjanje sporočil,**
 - **orkestracija/koreografija,**
 - **preoblikovanje sporočil**
 - **izvajanje poslovnih pravil.**
- Vzpostavitev takšne infrastrukture običajno zahteva **veliko truda**
 - **razvoj in konfiguracijo »pametnih« povezovalnih komponent.**
 - Pogosta kritika SOA je prav **visoka kompleksnost komunikacijske infrastrukture**, kar otežuje razvoj, spremembe in vzdrževanje.

52

Mikrostoritvena arhitektura: Pametne končne točke & neumne cevi

- V mikrostoritvah velja načelo:
pametne končne točke + preprosti komunikacijski kanali
(**smart endpoints, dumb pipes**)
- Storitve so čim bolj neodvisne, a povezne prek jasno opredeljenih vmesnikov.
- Vsaka storitev vsebuje svojo domensko (poslovno) logiko in obdeluje zahtevo po vzorcu:
prejme zahtevo → izvede logiko → pripravi odgovor.
- Povezovanje in »koreografija« med storitvami običajno poteka prek lahkih protokolov
 - npr. HTTP/REST, lahko tudi sporočil
 - Kompleksni standardi za orkestracijo/koreografijo (npr. BPEL, WS-* pristopi) se praviloma ne uporabljajo, ker povečajo kompleksnost.

53

Mikrostoritvena arhitektura: Pametne končne točke & neumne cevi

- Najpogostejša sta **dva načina/protokola komunikacije**
 - **HTTP (zahtev-odgovor):** storitev izpostavi **API nad viri** (pogosto REST)
 - **Lahka sporočilna komunikacija (asinhrono):**
 - uporablja se preprost **sporočilni posrednik** (message broker) oziroma vodilo, ki skrbi predvsem za **zanesljivo posredovanje sporočil**
 - Primeri: **RabbitMQ** (sporočilni posrednik), **ZeroMQ** (lahka sporočilna knjižnica).
- Načelo ostaja enako: **pametne so storitve** (proizvajalci in porabniki sporočil), komunikacijski kanal pa naj bo **čim preprostejši**.

54

Mikrostoritvena arhitektura: Decentralizirano upravljanje

- **Centralizirano upravljanje** pogosto zahteva **enotno, standardizirano tehnološko platformo**
 - Posledica je **manj svobode pri izbiri programskih jezikov, ogrodij in orodij**.
- **Mikrostoritveni pristop pogosto zagovarja decentralizacijo:**
 - Za vsako storitev izberemo tehnologijo, ki je za nalogo najprimernejša (polyglot programming / polyglot persistence).
 - npr. Node.js za spletne storitve,
 - C++ za rešitve z zahtevami po (skoraj) realnočasovni obdelavi,
 - različne sisteme za upravljanje podatkovnih zbirk (SUPB) glede na potrebe storitve.
- **Odnos do standardov je bolj pragmatičen:**
 - namesto obvezne enotnosti ekipe izbirajo rešitve, ki najbolje rešijo konkreten problem (ob upoštevanju dogovorjenih minimalnih smernic).

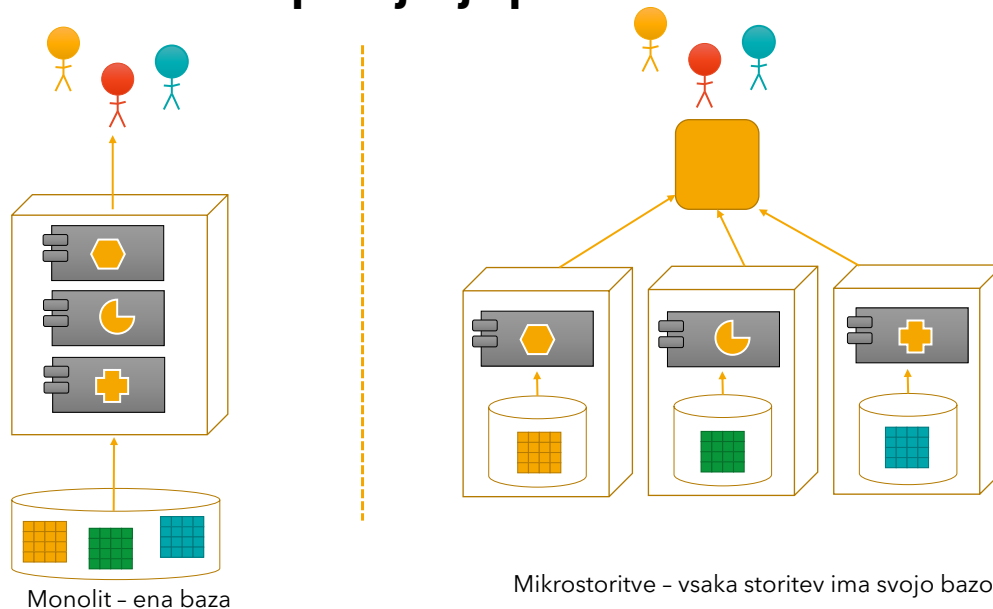
55

Mikrostoritvena arhitektura: Decentralizirano upravljanje podatkov

- Pri monolitnih sistemih je pogosto vzpostavljena ena skupna (logična) podatkovna baza, ki jo uporablja celotna aplikacija
- Mikrostoritve uvajajo decentralizacijo podatkov:
 - vsak del sistema ima svoj konceptualni podatkovni model,
 - storitve lahko uporabljajo različne načine shranjevanja podatkov.
- Praviloma velja načelo: **vsaka storitev je lastnik svojih podatkov in jih upravlja v svoji podatkovni bazi (ali drugi shrambi)**.
- To lahko pomeni:
 - ločene instance istega SUPB, ali
 - različne SUPB glede na potrebe storitve (polyglot persistence).

56

Mikrostoritvena arhitektura: Decentralizirano upravljanje podatkov



57

Mikrostoritvena arhitektura: Decentralizirano upravljanje tehničnih pogodb

- Decentralizacija prinese izzive pri upravljanju tehničnih pogodb (dogovorov o vmesnikih/API-jih) med storitvami.
- Uporabijo se vzorci, ki zmanjšujejo tveganje nezdržljivosti, npr.:
 - **Tolerant Reader** (odjemalec je odporen na spremembe v podatkih)
 - **Consumer-Driven Contracts** (pogodbe, ki jih sooblikujejo odjemalci storitev)
- Takšni vzorci omogočajo
 - hitrejšo povratno informacijo,
 - večjo zanesljivost integracije in
 - večje zaupanje, da storitve zagotavljajo pričakovane funkcionalnosti.

58

Mikrostoritvena arhitektura: **Avtomatizacija infrastrukture**

- Pri mikrostoritvah je **avtomatizacija infrastrukture v praksi nujna**, saj ročno upravljanje hitro postane neobvladljivo.
 - Razvoj oblačnih tehnologij je pomembno zmanjšal operativno zahtevnost gradnje, nameščanja in upravljanja mikrostoritev.
- **Rešitve se praviloma razvijajo z neprekinjeno integracijo (CI) in neprekinjeno dostavo (CD)**
 - Cevovodi (pipelines) omogočajo samodejno gradnjo, testiranje in uvajanje aplikacij v različna okolja (razvojno, testno, produkcijsko).

59

Mikrostoritvena arhitektura: **Načrt za neuspeh**

- Mikrostoritve morajo biti zasnovane tako, da **tolerirajo okvare** posameznih storitev (napake so pričakovane).
- Ker klic storitve lahko spodleti (npr. nedosegljiv ponudnik), se mora odjemalec ustrezno odzvati
 - časovne omejitve, ponovni poskusi, nadomestni odziv ipd.
 - Ključno je **hitro odkrivanje napak** in čim bolj avtomatizirano **obnavljanje storitev**.
- Zato mikrostoritvene rešitve močno poudarjajo sprotni nadzor (monitoring) in opazljivost:
 - **tehnične metrike** (npr. število zahtev na sekundo, odzivni časi, obremenitev podatkovne baze),
 - **poslovne metrike** (npr. število nakupov na sekundo, uspešnost transakcij).

60

Mikrostoritvena arhitektura: **Evolucijski načrt**

- Omogoča **hitre, pogoste in nadzorovane spremembe** brez zaviranja razvoja.
- Sistem razdelimo na **samostojne storitve** z jasnimi mejami odgovornosti.
- Ključ: storitve so **neodvisno nadgradljive in zamenljive**.
- Storitve lahko **prepišemo ali zamenjamo** z minimalnim vplivom na druge.
- V praksi se nekatere storitve sčasoma **opustijo in nadomestijo**.
- ***Mikrostoritve podpirajo evolucijski razvoj, ker omogočajo neodvisne spremembe, zamenjave in nadgradnje posameznih delov sistema.***

61

Mikrostoritve: prednosti

62

Join at menti.com | use code **6251 1071** Mentimeter

Prednosti mikrostoritev?

- lažje skaliranje
- Lažje vzdrževanje skozi leti
- ne potre se vse če ena storitev neha delati
- lažji razvoj za več ljudi hkrati
- Boljše skaliranja, lažje vzdrževanje
- Boljši vpogled v posamezne funkcionalnosti
- Razdelitev na manjše dele, lažja uporaba, fleksibilnost, lažja uporaba večih tehnologij, večja varnost zaradi porazdeljenosti
- Pri kompatibilnosti različnih storitev

← 🔍 → Next slide 4/15

63

Mikrostoritve: glavne prednosti

- Manjše in enostavnejše aplikacije
- Močne meje med moduli
- Neodvisen razvoj, nameščanje in skaliranje vsake storitve posebej
- Ni zahteve po dolgoročni zavezi sklada specifičnih tehnologij
- Tehnološka raznolikost in lažje preizkušanje tehnologij
- Boljša izolacija napak

64

Manjše in enostavnejše aplikacije

Boljša razumljivost

kodo in delovanje storitve je lažje razumeti kot pri velikem monolitu z zelo obsežno kodo.

Manj težav z odvisnostmi

manj konfliktov med knjižnicami in različicami (npr. v *classpathu*).

Hitrejša izgradnja in nameščanje

manj kode pomeni krajši čas prevajanja in hitrejši proces uvajanja.

Krajši čas zagona

storitve se pogosto zaženejo v sekundah, veliki monoliti pa lahko potrebujejo minute.

Višja produktivnost pri razvoju

hitrejši zagon in hitrejši cikli »spremeni-preveri« pospešijo delo razvijalcev.

65

Močne meje med moduli

- **Mikrostoritve okrepijo modularno zasnovo**, ker vsaka storitev predstavlja jasno omejeno funkcionalno celoto.
 - Tudi monolit lahko zgradimo modularno, vendar to pogosto zahteva večjo disciplino in dosledno spoštovanje mej med moduli.
- **Pri mikrostoritvah so meje bolj »prisiljene« z arhitekturo**
 - storitve se povezujejo prek API-jev, ne z neposrednim dostopom do notranje implementacije.
 - Na primer, pri decentraliziranem upravljanju podatkov lahko storitev do podatkov druge storitve praviloma dostopa le prek njenega API-ja in nima direktnega dostopa do njene podatkovne baze.

66

Neodvisno nameščanje

- Ključna ideja mikrororitev je, da so **storitve samostojne komponente**, zato jih lahko uvajamo neodvisno.
 - **Sprememba v eni storitvi praviloma zahteva ponovno uvedbo samo te storitve, ne celotnega sistema.**
- Zaradi avtonomnosti je tudi **zamenjava storitve (npr. nova implementacija) običajno lažja** kot zamenjava monolitne aplikacije.
- Neodvisno uvajanje zmanjšuje tveganje, da bi napaka v eni storitvi povzročila **izpad celotnega sistema**.
- V kombinaciji z neprekinjeno dostavo (CD) se skrajša čas od ideje do delujoče rešitve.

67

Ni zahteve po dolgoročni zavezi skladu specifičnih tehnologij

- Mikrororitve v teoriji omogočajo **poliglotski razvoj**
 - posamezne storitve lahko razvijamo v različnih **programskih jezikih** in z različnimi **ogrodji**.
 - Pri novi storitvi lahko (vsaj načeloma) izberemo tehnologijo, ki je za njen namen **najprimernejša**.
- Ko sistem raste in dodajamo nove storitve, nismo nujno vezani na tehnologije, s katerimi smo začeli – to prinaša več **prožnosti** pri uvajanju novih pristopov.
- Prednost je tudi pri kadrih
 - lažje vključimo nove člane ekipe, ki obvladajo **drugačen tehnološki sklad**.
- V praksi se pri večjih sistemih pogosto uvedejo **tehnološke smernice** (strateške odločitve), da se ohrani obvladljivost in zmanjšajo stroški vzdrževanja.

68

Tehnološka raznolikost in lažje preizkušanje tehnologij

- Mikrostoritve ekipam omogočajo **lažje uvajanje novih tehnologij**, saj lahko pri razvoju nove storitve izberejo drugačen tehnološki sklad.
 - Ker se tehnologije hitro razvijajo, je pomembno, da lahko sistem **postopno posodabljam**o in uvajamo rešitve, ki prinašajo napredek.
 - Zastarele tehnologije je mogoče **zamenjevati po korakih** (po storitvah), medtem ko je to pri monolitu pogosto bistveno zahtevnejše.
- Preizkušanje nove tehnologije je manj tvegano
 - če se izkaže za neprimerno, je vpliv praviloma **omejen na posamezno storitev** (izguba časa je obvladljiva, sistem pa ni ogrožen).
 - Tovrstni preizkusi prinesejo tudi **učenje** in hitrejše oblikovanje dobrih praks v ekipi.

69

Boljša izolacija napak

- V monolitni aplikaciji lahko resna napaka (npr. **poraba pomnilnika**) povzroči **izpad celotne rešitve**, ker vse teče v istem procesu.
- Pri mikrostoritvah se napaka praviloma omeji na **posamezno storitev**, saj se storitve izvajajo ločeno.
- Posledično lahko **preostali del sistema** še naprej deluje, medtem ko se prizadeta storitev obnovi ali začasno nadomesti (npr. z degradiranim delovanjem).

70

Prednosti mikrostoritvene arhitekture

- Enostavnejše dodajanje novih funkcionalnosti v obliki storitev
- Za razvoj novih storitev skrbijo ločene – neodvisne razvojne ekipe
- Neodvisnost od uporabljenih tehnologij pri razvoju posameznih storitev
- Enostavnejše testiranje zaradi manjšega obsega funkcionalnosti, razvitih v okviru posameznih storitev
- Napake v posameznih storitvah ne vplivajo na delovanje ostalih – neodvisnih storitev
- Storitve se hitreje zaženejo
- Neodvisno dodeljevanje virov posameznim storitvam
- Neodvisno nameščanje in posodabljanje storitev

71

Join at menti.com | use code 62511071
Mentimeter

Izzivi pri razvoju na osnovi mikrostoritev?

Usklajevanje med storitve

storitve v različnih regijah, zahteva po enotnih klicih

več se zapleta s pomočjo uporabe AI za razvoj

Povezat mikrostoritve

Konsistentnost podatkov

←
🔍
→ Next slide
4 / 15

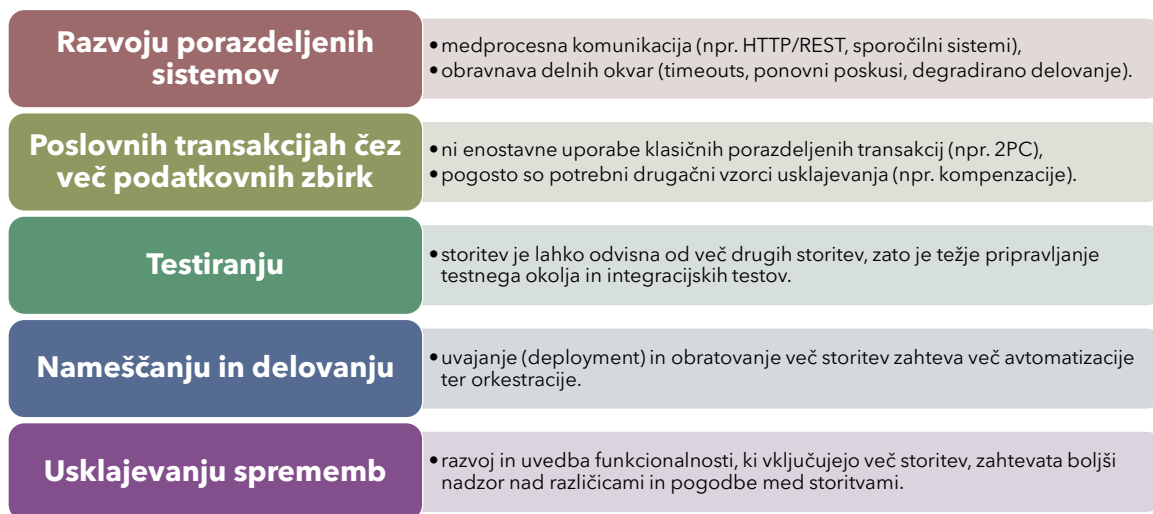
72

Izzivi pri razvoju na osnovi mikrorstitev



73

Večja **kompleksnost**, predvsem v



74

Več podatkovnih baz in upravljanje transakcij

- Pri mikroritvah ne razdelimo le aplikacije, temveč tudi **podatkovno shrambo**: vsaka storitev praviloma upravlja svojo podatkovno bazo.
- To poveča šibko sklopljenost in avtonomijo storitev, hkrati pa prinese dodatno kompleksnost.
- Poslovnih transakcij, ki vključujejo več storitev, običajno ne moremo več enostavno izvesti kot **ACID transakcije** prek enotne baze.
- Namesto tega uporabljamo drugačen programski model:
 - **dogodkovno usmerjeno obdelavo** in
 - **časoma konsistentne procese** (»eventual consistency«), pogosto z vzorci, kot so saga in kompenzacijske transakcije.

75

Kompleksnost testiranja porazdeljenega sistema

- Pri monolitnih aplikacijah so uveljavljeni pristopi, kot so **testi enot** in **avtomatizirano UI** testiranje (npr. Selenium), pogosto relativno preprosti za vzpostavitev.
- Pri mikroritvah želimo testirati storitev A, vendar je ta lahko odvisna od storitev B, C, ... (klici prek omrežja), zato se kompleksnost testiranja bistveno poveča.
- **Mikroritvena arhitektura zahteva dodatne strategije za avtomatizacijo**, npr. ločevanje med:
 - **testi enot**,
 - **testi komponent** (storitev + njene odvisnosti v kontroliranem okolju),
 - **testi pogodb** (contract testing),
 - **celostnimi testi** (end-to-end).
- Martin Fowler (Toby Clemson) predstavi pregled strategij testiranja mikroritev v članku Testing Strategies in a Microservice Architecture
<https://martinfowler.com/articles/microservice-testing/>

76

Kompleksnost nameščanja in delovanja porazdeljenega sistema

- V mikrostoritveni arhitekturi lahko sistem sestavlja **10, 50 ali 100+ samostojnih storitev**, vsaka s svojimi zahtevami in posebnostmi.
- Storitve so lahko razvite na različnih **platformah**, uporabljajo različne **različice ogrodij** in imajo različne odvisnosti.
- Uvajanje v produkcijo zato zahteva **orkestracijo** velikega števila delov (pri monolitu je to pogosto en sam namestitveni paket).
- Proces se običajno obvladuje z **avtomatizacijo uvajanja** (CI/CD) ter orodji za orkestracijo, kar bistveno poenostavi nameščanje in upravljanje.

77

Porazdeljenost: zmogljivost in zanesljivost

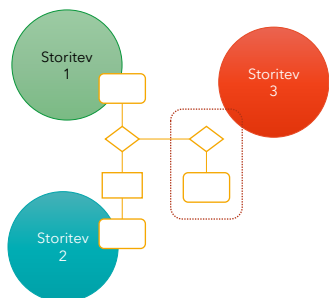
• Zmogljivost

- Klici znotraj procesa (in-process) v monolitu so praviloma **hitrejši** kot omrežni klici med storitvami.
- Klici oddaljenih storitev (inter-process) prinašajo **latenco**
 - veriga klicev ($A \rightarrow B \rightarrow C \dots$) lahko hitro poveča skupni odzivni čas.
- Zmanjševanje vpliva:
 - **dobro zasnovani vmesniki** (manj klicev, več podatkov na klic),
 - **asinhroni in vzporedni klici**, kjer je skupni čas še vedno omejen z najpočasnejšim klicem.

• Zanesljivost delovanja

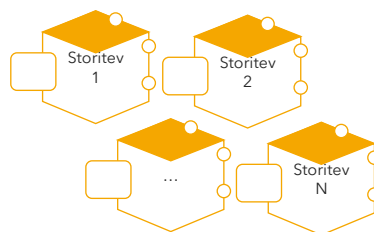
- Pri omrežnih klicih je vedno prisotno tveganje **nedosegljivosti** ali **časovnih prekoračitev**.
- Z večjim številom vključenih storitev se verjetnost delnih izpadov **povečuje**, zato je potrebna odpornost na napake.

78



**storitveno usmerjena
arhitektura**

VS



**mikrostoritvena
arhitektura**

79

SOA vs mikrostoritvena arhitektura

- Na spletu zasledimo veliko debat o razlikah med SOA in mikrostoritveno arhitekturo.
- Pristopa imata veliko skupnega: mikrostoritve so se razvile tudi na osnovi idej SOA.
- Oba pristopa temeljita na storitvah in spodbujata šibko sklopljenost ter jasno definirane vmesnike.
- Kljub podobnostim obstajajo tudi pomembne razlike v načinu integracije, upravljanja in uvajanja storitev.

80

Join at menti.com | use code 6251 1071 

Podobnosti med SOA in mikrostoritveno arhitekturo?

- SOA je nadpomenka
- Pomembnost dobro definiran vmesnik
- Delitev na manjše funkcijske enote
- Verzioriranje
- Obojni imajo locene /manjse/pouporabne storitve

← 🔍 → Next slide 4 / 15  

81

Join at menti.com | use code 6251 1071 

Razlike med SOA in mikrostoritveno arhitekturo?

- "mikro"
- SOA je bolj centralizirana in težja, mikrostoritve pa lažje, bolj razdrobljene in neodvisne.
- Usklajenost vmesnikov - med storitve se lahko razlikujejo

← 🔍 → End presentation 3 / 15  

82

Definicija (na spletu jih je ogromno)

SOA

Service-Oriented Architecture (SOA) is an architectural style that supports service-orientation. Service-orientation is *a way of thinking in terms of services and service-based development and the outcomes of services.*

A service:

- Is a logical representation of a repeatable *business activity* that has a specified outcome (e.g., check customer credit, provide weather data, consolidate drilling reports)
- Is *self-contained*
- May be composed of other services
- Is a "black box" to consumers of the service



<https://web.archive.org/web/20160819141303/http://opengroup.org/soa/source-book/soa/soa.htm>

Mikrostoritvena arhitektura

„In short, the microservice architectural style is an approach to *developing a single application as a suite of small services*, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around *business capabilities* and *independently deployable* by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.”

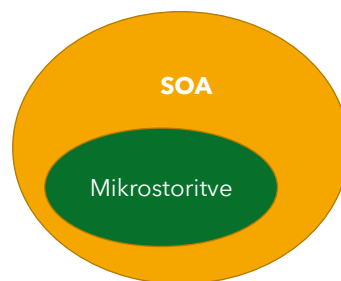
James Lewis and Martin Fowler

<https://martinfowler.com/articles/microservices.html>

83

SOA vs mikrostoritve

- Arhitekturni slog mikrostoritev bi lahko označili tudi kot specializacijo SOA
- SOA
 - zahteva jasno določene meje med storitvami,
 - storitve so avtonomne (razvijajo se in izvajajo relativno neodvisno)
 - storitve zagotavljajo pogodbe in sheme (jasno opredeljene vmesnike in podatkovne strukture)



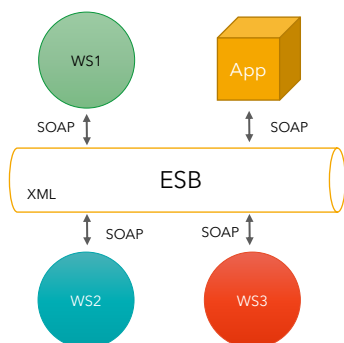
84

SOA vs mikrostoritvena arhitektura

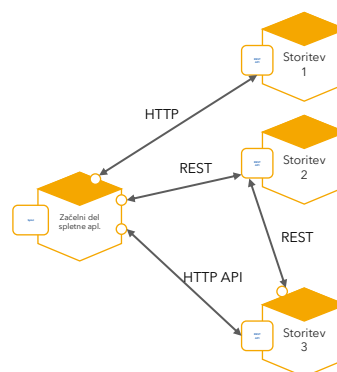
- Oba pristopa temeljita na gradnji **porazdeljenih sistemov iz storitev**
- Oba spodbujata **modularnost** in **šibko sklopljenost** ter v načelu omogočata večjo **neodvisnost od tehnologij**.
- Koncepti, ki jih danes povezujemo z mikrostoritvami, niso novi – pojavili so se že v času SOA (le v drugačni obliki in z drugačnimi praksami).
- Mikrostoritve poudarjajo razdelitev sistema na **manjše, jasno omejene module**
 - predvsem z namenom poenostavitve uvajanja (deploymenta), izvajanja in nadaljnjega razvoja.

85

SOA vs mikrostoritve



SOA aplikacije so sestavljene iz šibko sklopljenih komponent oz. storitev, ki komunicirajo preko storitvenega vodila (ESB) z izbranim komunikacijskim protokolom



Mikrostoritve so množica neodvisnih aplikacijskih storitev, ki ponujajo eno funkcionalnost na način, ki omogoča šibko povezanost in samo-vsebovanost, pri čemer komunicirajo z uporabo lahkih komunikacijskih protokolov kot so REST, HTTP, itd.

86

SOA vs mikrororitve – evolucija pristopa

- **Mikrororitve pogosto razumemo kot naslednji evlucijski korak storitveno usmerjenih arhitektur (SOA)**
 - z večjim poudarkom na sodobnih praksah razvoja in uvajanja
- Sistem (spletna ali mobilna aplikacija) je zgrajen kot skupek neodvisnih storitev, ki skupaj tvorijo celoto
 - **Posamezna storitev je zasnovana okoli ene poslovne zmožnosti** (npr. upravljanje uporabnikov, upravljanje zalog, ...)
- Storitve naj bodo čim bolj avtonomne, kar omogoča:
 - **neodvisno uvajanje (deployment) in skaliranje**
 - po potrebi **uporabo različnih programskih jezikov in ogrodij** (poliglotski razvoj)

87

SOA: glavne lastnosti

- V definiciji SOA je **velik pomen na orkestraciji storitev**
 - **Če v arhitekturi SOA ni orkestracije, pomeni, da v bistvu nimamo SOA**
 - **Zahteva po orkestraciji storitev je v nasprotju temu, kar zahteva mikrororitvena arhitektura (šibka sklopljenost)**
 - Orkestracija storitev zahteva določen nivo sklopljenosti storitev, ki pa ga v mikrororitveni arhitekturi ne želimo
- SOA hkrati zahteva nadzor oz. centralizirano upravljanje storitev (governance)

88

Mikrostoritve: glavne lastnosti

- Sistem je sestavljen iz **šibko sklopljenih storitev** z minimalno centraliziranega upravljanja.
- Vsaka storitev se izvaja kot **samostojen proces** (pogosto v več instancah).
- Storitve so zasnovane okoli **poslovnih zmožnosti** in imajo **jasno, ozko odgovornost**.
- »Mikro« se ne nanaša na število vrstic kode, temveč na **omejen obseg odgovornosti storitve**.
- Storitve je mogoče **uvajati in zaganjati neodvisno** od drugih storitev.
- **Razvojne ekipe** so pogosto organizirane po storitvah in so **avtonomne** pri izbiri tehnologij ter načinu razvoja.
- Pogosta je **uporaba načel DevOps**
 - neprekinjena integracija (CI),
 - avtomatizirano testiranje in
 - avtomatizirano uvajanje (CD).

89

SOA vs mikrostoritve

Centralizirano upravljanje

- Ena izmed ključnih razlik je v stopnji centraliziranega upravljanja.
- Pri SOA je pogosto prisoten večji poudarek na strogo centraliziranem upravljanju (standardizacija, skupne platforme, centralna integracija).
- Pri mikrostoritvah je centralizacija praviloma minimalna - **poudarek je na decentraliziranem upravljanju storitev in pogosto tudi podatkov**.
- To ekipam omogoča več avtonomije, hkrati pa zahteva boljšo avtomatizacijo in jasne tehnične dogovore (npr. pogodbe API).

90

SOA vs mikrostoritve

Neodvisne storitve in orkestracija storitev

- **Mikrostoritve so praviloma avtonomne, šibko sklopljene storitve**, ki vsebujejo lastno domensko logiko in komunicirajo prek preprostih vmesnikov.
 - Fowler in Lewis to definirata kot koncept **pametnih končnih točk** in **neumnih cevi**
- SOA je pogosteje usmerjena v orkestracijo in centralizirano koordinacijo storitev.
- V SOA storitve pogosto komunicirajo prek skupnega integracijskega mehanizma
 - na primer storitvenega vodila (Enterprise Service Bus – ESB), ki lahko prevzame del logike usmerjanja, preoblikovanja in povezovanja.

91

SOA vs mikrostoritve

Neodvisnost in način povezovanja

- **SOA:** storitve pogosto komunicirajo prek skupnega integracijskega vodila (ESB) in so bolj odvisne od centraliziranega upravljanja.
 - Za interoperabilnost je potreben širši dogovor o vmesnikih – pogosto tudi o formatih sporočil in pravilih integracije.
- **Mikrostoritve:** storitve se izvajajo v lastnem izvajalnem okolju in se praviloma razvijajo ter uvajajo neodvisno.
 - Integracija je običajno zasnovana prek lahkih API-jev in/ali sporočilne komunikacije, brez "pametnega" centralnega vodila.

92

SOA vs mikrostoritve

Standardizacija vs. avtonomija storitev

- Tako pri SOA kot pri mikrostoritvah je mogoče **storitve razvijati z različnimi programskimi jeziki in orodji**.
 - Razvoj je lahko razdeljen med več razvojnih ekip.
- **SOA:** ekipe se morajo **uskladiti glede skupnega načina komunikacije** (protokoli, tehnologije, formati sporočil), kar pogosto poveča odvisnost od skupne infrastrukture.
- **Mikrostoritve:** **poudarjajo avtonomnost storitev**, kar omogoča
 - pogosto uvajanje novih različic posameznih storitev,
 - neodvisno skaliranje storitev glede na obremenitev.

93

SOA vs mikrostoritve

Podatki

- **SOA:** storitve si lahko delijo **skupno podatkovno plast** oziroma isti sistem za upravljanje podatkovne baze (SUPB), ki ga uporablja več storitev.
 - Storitve so lahko še vedno šibko sklopljene in se izvajajo samostojno, vendar je **podatkovna plast pogosto skupna**.
- **Mikrostoritve:** poudarek je na **neodvisnem upravljanju podatkov** (database per service).
 - Praviloma je **vsaka storitev lastnik svojih podatkov in si ne deli iste podatkovne baze z drugimi storitvami** (dostop poteka prek API-jev).

94

SOA vs mikrostoritve – ključne razlike

Vidik	SOA	Mikrostoritvena arhitektura
Arhitekturni poudarek	Deljenje poslovnih zmožnosti prek medsebojno povezanih storitev, pogosto ob podpori skupne integracijske infrastrukture.	Samostojne storitve z jasno, ozko odgovornostjo; storitve se razvijajo in uvajajo neodvisno.
Deljenje komponent	Pogosteje vključuje deljenje skupnih komponent/knjižnic, kar lahko poveča tesno sklopljenost .	Praviloma brez deljenja komponent; vsaka storitev ima svoje komponente in odvisnosti.
Podatkovna shramba	Lahko uporablja skupno podatkovno shrambo/SUPB za več storitev, kar lahko povzroči odvisnosti.	Načelo podatki po storitvah : vsaka storitev upravlja svojo shrambo, dostop poteka prek API.
Upravljanje in standardi	Pogosteje centralizirano upravljanje (standardizirani protokoli, pravila integracije).	Manj centralizacije, več avtonomije ekip ; poudarek na pogodbah API in avtomatizaciji.
Nameščanje in skaliranje	Pogosto bolj usklajeno/centralno; spremembe lahko vplivajo na širši ekosistem integracij.	Neodvisno uvajanje in neodvisno skaliranje posameznih storitev.
Tipični kontekst uporabe	Večje integracije in heterogena poslovna okolja, kjer je interoperabilnost ključna.	Okolja s pogostimi spremembami in potrebo po hitri dostavi ter prilagodljivem skaliranju (pogosto spletni sistemi).

95

SOA vs mikrostoritve – ključne razlike

Vidik	SOA	Mikrostoritvena arhitektura
Komunikacija	Pogosto uporablja storitveno vodilo (ESB) kot osrednji integracijski mehanizem.	Komunikacija poteka prek lahkih API-jev ali sporočilne komunikacije brez "pametnega" centralnega vodila.
Tipični protokoli	Pogosto SOAP ; sporočilni protokoli se lahko uporabljajo v okviru integracijske infrastrukture (npr. AMQP).	Pogosto REST/HTTP ; za asinhrono sporočanje se uporabljajo posredniki/ogrodja (npr. JMS, AMQP).
Oddaljene storitve	Pogosteje uporablja težje/bolj formalne standarde in pogodbe, kar lahko poveča kompleksnost integracije.	Praviloma uporablja preprostejše protokole in jasne API pogodbe, kar olajša integracijo in razvoj.
Nameščanje (uvajanje)	Pogosto manj prožno, ker spremembe lahko zahtevajo več usklajevanja in vplivajo na širši ekosistem integracij.	Neodvisno uvajanje posameznih storitev brez neposrednega vpliva na celoten sistem (če so pogodbe stabilne).
Fleksibilnost in skalabilnost	Skaliranje je lahko oteženo zaradi centralizirane infrastrukture in odvisnosti med storitvami.	Enostavnejše skaliranje po storitvah ; visoka prožnost pri prilagajanju obremenitvam.
Tehnološka usmeritev	Pogosteje povezano s tradicionalnimi podjetniškimi tehnologijami in močno standardizacijo.	Pogosto povezano s sodobnimi praksami: zabojniki (Docker), orkestracija (Kubernetes), DevOps, CI/CD .

96